

Report - Assignment 3

Date of submission : 14/11/24

Vaibhav Ahuja 2022EE11839

Pratyush Agarwal 2022EE11158

Aryan Verma 2022EE11671

Objective function -

1. **Final Submission** - Used *CrossEntropy* loss
2. The selected model (**ResNet**) are suited for CrossEntropy loss, as said in the paper, hence went for it. Tried out MSE loss also, the results were not very satisfactory
3. Reasons why *CrossEntropy* loss was best suited -
 - a. Outperforms other optimization techniques used with dense net (**source - [1]**)
 - b. Highly effective for multi-class classification tasks like CIFAR-100. The cross-entropy loss function works by measuring the difference between the predicted probability distribution and the true class label.

Model Architectures

We experimented with several architectures, including ResNet-18[3], ResNet-50[3], WideResNet-50_2, EfficientNet-B0, [5]DenseNet-121, 169, and 201. After testing these models, we selected **DenseNet-121** as the final architecture due to its superior performance and efficiency.

- **DenseNet-121** was chosen because it:
 - Promotes feature reuse through dense connections, ensuring efficient use of parameters and better gradient flow.
 - Has fewer parameters than deeper models like ResNet-50 or DenseNet-169, while still achieving high accuracy, making it suitable for constrained resources.
 - Demonstrated faster convergence and produced more reliable, high-confidence predictions.

DenseNet-121's balance between parameter efficiency and accuracy made it the optimal choice for this project.

Model Details-

The DenseNet-121 architecture is a type of convolutional neural network (CNN) known for its efficiency and ability to facilitate **feature reuse** through dense connections. Each layer in DenseNet receives inputs from all preceding layers, leading to enhanced gradient flow and a **reduction in the vanishing gradient problem**, which is particularly advantageous in deep networks.

1. **Input Layer:** The model begins with a convolutional layer (**conv0**), which operates on 3-channel input images (32x32x3). This layer uses a 3x3 kernel and 64 filters, followed by batch normalisation (**norm0**), a ReLU activation function (**relu0**), and an identity mapping for pooling (**pool0**).
2. **Dense Blocks:** The architecture contains four dense blocks. Each block comprises multiple dense layers that are connected in a feed-forward manner, where each layer takes input from all preceding layers. This allows for a high degree of feature reuse, making the model both parameter-efficient and capable of learning complex patterns with fewer layers.
 - Each dense layer includes a pair of convolutional operations:
 - A 1x1 convolution that increases the channel depth.
 - A 3x3 convolution that extracts feature maps, followed by batch normalisation and ReLU activation.
 - As layers are added, the depth of the feature maps increases, ensuring that the model captures diverse representations.
3. **Transition Layers:** After each dense block, transition layers reduce the spatial dimensions of the feature maps using 1x1 convolutions and average pooling. These layers play a critical role in controlling model complexity and preventing the feature maps from becoming overly large.
4. **Final Layers:** Following the fourth dense block, the network passes through a final batch normalisation layer (**norm5**) before the classifier. The classifier is a fully connected layer that outputs predictions for 100 classes, making this architecture suitable for CIFAR-100 classification tasks.
5. **Key Strengths:**
 - **Dense Connections:** These connections facilitate efficient feature reuse, reducing the number of parameters and improving gradient propagation.
 - **Batch Normalization and ReLU Activation:** These layers after every convolution ensure stable training and faster convergence by normalizing the activations and introducing non-linearity.

DenseNet-121 is highly regarded for its balance between model depth and efficiency, making it a great choice for tasks where computational resources are limited but accuracy is still a priority.

Summary of Dense Block I/O:

1. **Dense Block 1:**
 - Input: 64 channels.
 - Output: 256 channels (6 layers).
2. **Dense Block 2:**
 - Input: 128 channels.
 - Output: 512 channels (12 layers).
3. **Dense Block 3:**
 - Input: 256 channels.
 - Output: 1024 channels (24 layers).
4. **Dense Block 4:**
 - Input: 512 channels.
 - Output: 1024 channels (16 layers).

Detailed Flow:

1. **Initial Conv Layer:** 3-channel image $\rightarrow$ 64 channels.
2. **Dense Block 1:** 64 $\rightarrow$ 256 channels (6 layers).
3. **Transition 1:** 256 $\rightarrow$ 128 channels.
4. **Dense Block 2:** 128 $\rightarrow$ 512 channels (12 layers).
5. **Transition 2:** 512 $\rightarrow$ 256 channels.
6. **Dense Block 3:** 256 $\rightarrow$ 1024 channels (24 layers).
7. **Transition 3:** 1024 $\rightarrow$ 512 channels.
8. **Dense Block 4:** 512 $\rightarrow$ 1024 channels (16 layers).
9. **Final BatchNorm + Classifier:** 1024 $\rightarrow$ 100 classes.

Justification of Approach and Experiments

Why Dense Net 121 :

- We began by testing several well-known deep learning architectures that have been successful in image classification tasks, including **ResNet-18**, **ResNet-50**, **WideResNet-50_2**, **EfficientNet-B0**, and **DenseNet** models (121, 169, 201).
- **ResNet-18** and **ResNet-50**: These architectures are known for their residual connections, which help avoid the vanishing gradient problem. While ResNet-18 performed adequately with faster training times, it did not achieve the desired accuracy on CIFAR-100 due to its shallower depth (Lesser predictions for threshold and more incorrect predictions). ResNet-50

improved performance but at the cost of increased parameters and longer training times which also made the model overfit to the training data (hence changed the model to find if there is a better alternative).

- **WideResNet-50_2**: This variation with a wider architecture showed better results than ResNet-18 due to its higher capacity, but it still had issues with overfitting, which was reflected in the larger gap between training and validation accuracies.
- **EfficientNet-B0**: EfficientNet-B0, known for its high efficiency, provided faster convergence due to lesser parameters but accuracy was not very good.

Outcome: After comparing these models, **DenseNet-121** provided the best balance between performance, parameter efficiency, and training time. Its dense connections promote feature reuse, enabling better gradient flow and learning with fewer parameters, making it well-suited for our task.

Feasibility of Dense Net 121 :

Enhancing Information Transmission: Traditional convolutional networks connect layers sequentially, with each layer receiving input only from the previous one, which can limit information flow and lead to issues like the vanishing gradient problem in deeper networks. Dense blocks address this by creating direct connections between each layer and all subsequent layers, allowing every layer to access feature maps from all previous ones, thus improving feature representation and gradient flow across the network.

Mitigating the Vanishing Gradient Issue: As networks grow deeper, gradient signals can diminish, slowing learning and hindering training. Dense blocks address this by providing multiple direct paths for gradients to propagate, allowing them to flow freely across the network. This ensures deeper layers benefit from gradients originating earlier, making training very deep networks more manageable without the vanishing gradient problem.

Promoting Feature Map Reuse: In conventional architectures, each layer learns a distinct set of features, often leading to redundancy as similar patterns are relearned by different layers. Dense blocks eliminate this redundancy by reusing the feature maps from all preceding layers. Each layer can, therefore, build upon the collective knowledge of earlier layers, making the network more efficient in terms of parameters while avoiding unnecessary repetition in feature learning.

Parameter Efficiency: Although dense connections may seem like they would increase the number of parameters, they actually lead to more efficient use of them. By reusing feature maps from prior layers, the network avoids the need to relearn redundant features, thus reducing the overall number of parameters. Consequently, DenseNets are more compact and require fewer parameters compared to architectures like ResNet, all while maintaining or even improving performance.

Implicit Deep Supervision: Dense blocks also provide a form of inherent deep supervision. Since each layer can directly access both the original input and the gradients from the loss function, it helps each layer learn meaningful representations. This implicit supervision ensures that each layer is better guided during the training process, especially as the network becomes deeper, leading to more effective learning in deeper layers without solely relying on intermediate transformations.

Facilitating Gradient Flow and Simplifying Training: The dense connectivity structure improves the flow of gradients throughout the network, leading to more stable training and faster convergence. With multiple pathways for information to travel, gradients from the loss function can reach earlier layers more easily, making the network easier to optimise compared to other very deep architectures. This smoother gradient flow results in more efficient training overall

Data Preparation

1. Preprocessing Methods:

- **Normalisation:**

- The images are normalised using the mean and standard deviation of the CIFAR-100 dataset. This ensures that each channel of the image has a zero mean and unit variance, which helps in stabilising the training process and ensures faster convergence.

2. Augmentation Techniques:

- **Random Crop with Padding:**

- A padding of 4 pixels is added on each side of the image, increasing the size from 32×32 to 40×40 . From this padded image, a random crop is taken to bring it back to 32×32 . This technique helps make the model more robust to small shifts and distortions in the data.

- **Random Horizontal Flip:**

- The image is randomly flipped horizontally with a 50% probability. This technique introduces variance into the training data and helps the model generalise better to flipped versions of the same image, making it invariant to horizontal flips.

3. Dataset Splitting Strategy - Training and Validation Splits:

- In this context, the CIFAR-100 dataset is split into:

- **Training Set:** Contains the majority of the data (**90%** of the dataset) used for learning.
- **Validation Set:** A smaller portion of the data (**10%** of the dataset) held out to assess the model's performance during training to monitor overfitting. The validation set was **balanced** to have equal images from each class

Training Methodology

1. **Model Architecture:**

- We used a standard deep learning architecture suitable for image classification tasks. The model was trained on the CIFAR-100 dataset, and its complexity was chosen based on the capacity required to achieve high accuracy for this multi-class classification task.

2. **Hyperparameters:**

- **Batch Size:** We used a batch size of 128, which provided a good balance between convergence speed and memory utilisation.
- **Learning Rate:** Several learning rates were tested, ranging from $1e-3$ to $1e-4$. The best results were obtained with a learning rate of $5e-4$, providing stable convergence without overshooting.
- **Epochs:** The total number of training epochs was set to 60. Early stopping was applied to prevent overfitting by halting the training if the validation performance did not improve within a set patience threshold.

3. **Loss Function Optimization:**

- **Loss Optimization:** The learning process involves monitoring the training and validation loss to ensure proper convergence. It was optimised using stochastic gradient descent (SGD) with momentum for better convergence. We adjusted the learning rate based on validation performance, and used early stopping to terminate training when the loss did not decrease further, preventing unnecessary overfitting and reducing computation time.

4. **Regularisation Techniques:**

- **L2 Regularization (Weight Decay):** L2 regularisation with a penalty of $\lambda=5e-4$ was applied to the model. This technique helps prevent overfitting by penalising large weight values. L2 regularisation was particularly useful in this case, as we observed a more significant than expected difference between training and testing accuracies.
- **Early Stopping:** To prevent overfitting and reduce computation time, early stopping was used. The model training was halted when the validation loss stopped improving, ensuring we only kept the best-performing model.

Inference :

* Prediction Methodology: Describe the inference process, detailing how predictions are generated and any postprocessing steps applied. * Confidence Threshold: If applicable, explain how a confidence threshold was used to filter predictions, including reasoning behind the chosen threshold value.

Inference Process:

- During the inference phase, each test image is first preprocessed using a set of transformations (e.g., normalization with dataset-specific means and standard deviations).
- The preprocessed images are then passed through the modified DenseNet121 model to generate output logits, which represent unnormalized class scores.
- These logits are converted into class probabilities using the softmax function, ensuring that the sum of the probabilities over all classes equals 1.

Post-Processing:

- Once the probabilities are computed, we apply a confidence thresholding mechanism. The model selects the top two class probabilities for each image. If the highest probability exceeds a confidence threshold of 0.9, the class corresponding to this probability is chosen as the predicted label.
- If the highest probability is below 0.9, we set the prediction to **-1**, indicating that the model is not confident enough to make a prediction for that sample. This thresholding reduces the chance of low-confidence or incorrect predictions affecting the model's performance.
- This approach is particularly useful in scenarios where incorrect predictions can have significant consequences, allowing us to only retain highly reliable classifications.

Confidence Threshold:

- The confidence threshold of 0.9 was not arbitrarily chosen but rather tuned through cross-validation on the trained model. We evaluated several threshold values during the validation process, and 0.9 was found to strike a good balance between retaining enough predictions and ensuring a high level of certainty in those predictions.
- The reasoning behind choosing 0.9 is as follows:

- **Higher Accuracy:** Predictions below 0.9 had a higher chance of being incorrect, so filtering them out led to an overall improvement in model accuracy.
- **Reducing False Positives:** By setting a relatively high threshold, we ensure that only predictions with a strong confidence level are considered, minimising the likelihood of false positives.
- **Empirical Results:** Through cross-validation, we observed that a threshold of 0.9 provided the best trade-off between accuracy and prediction coverage.

Cross-Validation Process:

- Cross-validation was performed using a **Stratified Shuffle Split** method to ensure the validation sets maintained the same class distribution as the training data. The dataset was split into training and validation sets, with 4000 examples set aside for validation (10% of the total data).
- The validation process involved evaluating different confidence thresholds (ranging from 0.7 to 0.95) and selecting the one that optimised model performance on unseen data.
- we specifically monitored the model's ability to maintain high precision (correctness of predictions) and recall (ability to make predictions on most samples). After this tuning process, 0.9 emerged as the most effective threshold.

Batch Size and Performance Optimization:

- The model was trained using a batch size of 128, and the Adam optimizer was used with a learning rate of 5e-4 and weight decay of 5e-4 for regularisation. These hyperparameters were chosen to prevent overfitting and to ensure smooth convergence.
- To improve performance during training and inference, we leveraged **DataLoader** optimizations, such as `pin_memory` and multi-threading (`num_workers=4`). This allowed for faster data loading, ensuring that the GPU remains fully utilised during training.
- The model training loop includes early stopping to save the best model based on validation loss, further ensuring that overfitting is minimised, and the best generalising model is used for prediction.

Report_2 - Assignment 3

Date of submission : 20/10/24

Vaibhav Ahuja 2022EE11839

Pratyush Agarwal 2022EE11158

Aryan Verma 2022EE11671

New Models Tried

Experimentation with Efficient Neural Network Architectures and Optimizers

In this phase of the project, We experimented with newer and more compact neural network architectures such as **ShuffleNet**, **VGG 16**, **Residual Attention Net** and finally **GoogLeNet**. These models are designed to be both efficient and lightweight, making them ideal for deployment in environments with limited computational resources. Their architectural innovations allow for reduced parameter counts and faster inference times without significantly compromising accuracy.

Additionally, we explored various optimization algorithms to enhance the training process, including **Stochastic Gradient Descent (SGD)** and **Adam**. Alongside different optimizers, we experimented with varying batch sizes and learning rates to assess their impact on model convergence and performance. The goal was to identify configurations that improved training speed and overall model generalisation on unseen data.

This combination of efficient architectures, diverse optimizers, and varying training parameters was crucial in identifying the optimal setup that balances performance and resource utilisation.

Optimization Strategies and Batch Size Experiments

We experimented with various optimization algorithms, primarily **Stochastic Gradient Descent (SGD)** and **Adam**. **SGD** proved to be a significant factor in improving generalization. With this optimizer, the validation accuracy increased from 65% (using DenseNet) to 70% (using GoogLeNet), indicating better performance on unseen data. This improvement also led to an enhanced Kaggle score, which rose from **4500** to **5600**.

Alongside the choice of optimizer, batch sizes were also carefully tuned. Initial experiments with smaller batch sizes, such as **16** and **32**, demonstrated faster but less stable training. While training times were shorter, the models tended to converge prematurely or oscillate, which limited their effectiveness. To address this, we increased the batch size to **128** and **200**, which stabilized the training process and improved training efficiency.

Adjustments in Training Epochs

Given that **GoogLeNet** has significantly fewer parameters than **DenseNet**, we extended the training duration to **100 epochs**. This decision allowed the model to reach optimal convergence, making full use of its capacity while ensuring better generalisation and improved validation accuracy.

Additional Experimentation with Temperature Scaling

We also explored **temperature scaling** as a post-processing technique to calibrate model confidence. However, this approach did not yield significant improvements in performance or confidence calibration, suggesting that it might not be well-suited for this particular model setup.

Class-wise Confidence Thresholding:

One of the innovative strategies implemented was **class-wise probability thresholding**. This approach involves setting a specific probability threshold for each class: if the predicted probability for a given class is p , and the threshold is t , the model only classifies the instance as that class if $p > t$. If p is below the threshold, the model refrains from making a confident prediction, indicating uncertainty in its classification. This technique provided a more nuanced decision-making process, allowing for better handling of low-confidence predictions.

In the initial approach, we applied a common confidence threshold across all classes for making predictions. However, we observed that certain classes were consistently predicted with lower confidence, while others performed better. This led us to introduce a more tailored approach by determining class-specific confidence thresholds.

To achieve this, we identified a threshold for each class that resulted in at least 80% accuracy for that class. This adjustment ensured that the accuracy for nearly all classes exceeded 70%. Since the validation set was relatively small, we opted for higher thresholds to maintain a robust and reliable prediction performance.

By adopting this class-wise thresholding strategy, we balanced the prediction confidence across all classes, leading to more consistent results across the entire dataset.

Experimented with:

1. VGG16:

- **High parameter count:** VGG16 is a deep architecture with 16 layers, but it uses simple 3x3 convolutions, which lead to a very high number of parameters. While VGG16 is powerful for tasks like ImageNet, it struggles on smaller datasets like CIFAR-100 due to overfitting. The large parameter count causes the model to memorize the training data instead of generalizing well on the validation set.
- **Lack of advanced features:** VGG16 lacks modern design elements like residual connections, which means it doesn't effectively mitigate the

vanishing gradient problem in deeper networks, making training less efficient.

2. NasNet:

- **Search complexity:** NasNet models are designed using neural architecture search, which aims to find optimal architectures automatically. However, these models tend to be highly optimized for the dataset they were searched on (usually ImageNet). When applied to different datasets like CIFAR-100, the learned architecture may not generalize well, leading to suboptimal performance.
- **High computational cost:** The architecture of NasNet is quite complex and computationally expensive, making it prone to overfitting on smaller datasets like CIFAR-100, where the model might not be able to fully exploit its potential.

3. Residual Attention Network:

- **Attention mechanisms:** While attention mechanisms can significantly enhance the focus on important regions of an image, they add extra complexity to the network. Residual Attention Networks might overfit small datasets due to their intricate design, especially if the attention maps aren't able to focus effectively on smaller objects or fine details in CIFAR-100.
- **Overfitting tendency:** The added complexity from attention layers increases the model's capacity, making it more prone to overfitting unless regularization techniques or data augmentation are used more aggressively.

4. ShuffleNet:

- **Designed for mobile efficiency:** ShuffleNet was designed for mobile devices, prioritizing model size and inference speed over accuracy. While it performs well in low-resource settings, it often trades off accuracy for efficiency, and this may be a reason for its lower performance on CIFAR-100, which requires a more robust feature extraction process.
- **Limited feature extraction capability:** The architecture uses grouped convolutions and channel shuffling to reduce computation, which can limit the model's ability to extract rich, detailed features from small images like those in CIFAR-100.

These models did not give good accuracies (max was for NasNet which is 62%, lesser than our previous submission). Hence these, We decided to take GoogleNet.

GoogleNet

Also known as **Inception v1**, is a significant milestone in the evolution of convolutional neural network architectures. Introduced by researchers at Google in 2014, it was designed with a strong focus on both **accuracy and computational efficiency**, addressing challenges related to parameter optimization and scalability in large-scale image classification tasks.

1. Inception Module: Multi-Scale Feature Extraction

- The most distinctive feature of GoogLeNet is the **Inception module**, which allows the network to capture a diverse set of features at multiple scales simultaneously.
- This module applies different filter sizes (e.g., 1x1, 3x3, 5x5) in parallel to the same input, enabling the model to identify a wide range of patterns, from small details to broader textures, all within a single layer.
- The introduction of **1x1 convolutions** before larger filters is particularly noteworthy, as it reduces the input's dimensionality, making the network more efficient.
- This multi-scale processing contributes to improved accuracy while maintaining computational efficiency.

2. Depth and Width Balance: Structuring the Network

- GoogLeNet maintains a balance between depth and width by strategically stacking Inception modules. The network achieves significant depth (22 layers) while maintaining relatively low computational costs.
- Unlike purely deep architectures, GoogLeNet leverages parallel convolution paths to maintain feature richness while controlling the number of parameters.
- Instead of ending with fully connected layers, the model uses **average pooling**, which reduces the risk of overfitting and keeps the parameter count low.

3. Efficient Use of Parameters: Lower Computational Costs

- A key aspect of GoogLeNet's design is its efficiency in parameter usage. Compared to earlier dense models, GoogLeNet has significantly fewer parameters—around **5 million**—which is a fraction of the parameter count found in models like VGG16.

- This reduction makes the model faster, less memory-intensive, and better suited for real-time applications or resource-constrained environments.
- Despite having fewer parameters, GoogLeNet maintains competitive performance, thanks to the use of multi-scale feature extraction and dimensionality reduction techniques.

4. Auxiliary Classifiers: Enhancing Gradient Flow

- To address the issue of **vanishing gradients**, especially in deeper layers, GoogLeNet incorporates auxiliary classifiers at intermediate layers.
- These auxiliary networks provide additional supervision during training, acting as both regularizers and support for gradient propagation.
- By assisting the backpropagation process, they help earlier layers learn more effectively, improving convergence and performance.

5. Strengths and Advantages of GoogLeNet

- **Efficient Multi-Scale Learning:** The architecture excels at identifying both fine details and broader patterns within images, leading to better overall feature extraction.
- **Reduced Parameter Count:** By effectively managing the number of parameters through techniques like average pooling and dimensionality reduction, GoogLeNet achieves a balance between performance and computational cost.
- **Improved Generalisation:** Despite its complex architecture, GoogLeNet demonstrates strong generalisation capabilities, making it a reliable choice for image classification tasks.

6. Limitations of GoogLeNet

- **Architectural Complexity:** The structure of the Inception module adds complexity to implementation and debugging, making it less straightforward than simpler architectures like VGG.
- **Limited Applicability to Sequential Tasks:** GoogLeNet's design is tailored for image classification tasks, making it less suitable for sequential data, where context-aware architectures like transformers are more effective.
- **Lack of Residual Connections:** Unlike later models such as ResNet, GoogLeNet lacks residual connections, which could limit gradient propagation in even deeper networks.

Report_3 - Assignment 3

Date of submission : 20/10/24

Vaibhav Ahuja 2022EE11839

Pratyush Agarwal 2022EE11158

Aryan Verma 2022EE11671

New Models Tried

Model Architecture Overview

The experimental model was built upon GoogLeNet architecture, enhanced with attention network features. The key architectural modifications included:

- Integration of channel attention mechanisms
- Addition of spatial attention layers
- Retention of GoogLeNet's inception modules
- Custom attention feature integration

Implementation of Attention Mechanisms

Channel Attention

- Implemented to weigh the importance of different feature channels
- Helped in focusing on most relevant feature maps
- Utilized squeeze-and-excitation blocks for channel relationship modeling
- Aimed to capture channel-wise dependencies

Spatial Attention

- Designed to focus on specific regions of interest in feature maps
- Implemented using convolutional layers for spatial relationship learning
- Enhanced the model's ability to focus on relevant spatial locations
- Integrated after channel attention for hierarchical feature refinement

Performance Analysis

Improvements

- Achieved a 2% increase in overall accuracy compared to baseline
- Enhanced feature representation through attention mechanisms
- Showed potential in capturing complex feature relationships

Critical Issues and Limitations

Prediction Confidence

- Model exhibited low confidence in predictions
- Uncertainty in classification decisions
- Inconsistent prediction patterns across similar inputs

Scaling Challenges

- Unable to maintain performance improvements at scale
- Computational overhead increased significantly
- Resource utilization inefficient for production deployment

Attempted Solutions

Monte Carlo Dropout Implementation and Test Time Augmentation (TTA) -

They failed to give reliable results

Conclusion

Despite the theoretical advantages of attention mechanisms and the modest accuracy improvement, the model's practical limitations outweighed its benefits. The key issues of low prediction confidence and scaling challenges make this implementation unsuitable for production deployment. The attempted remediation through Monte Carlo Dropout and TTA did not yield satisfactory results, suggesting the need for a different architectural approach.

.

Monte Carlo Dropouts

GoogleNet: Uncertainty Estimation using Monte Carlo Dropout

We used MC Dropout in conjunction with a GoogleNet architecture to estimate the uncertainty of predictions for this project. Enabling dropout layers during inference, enabled us to sample from a distribution of predictions for each input example and we utilized this variability to analyze the model's confidence in its predictions.

Method and Implementation

In GoogleNet, we used MC Dropout with 30/100 forward passes per input and dropout layers active. That generated 30/100 slightly different predictions for each sample allowing to calculate our two main metrics:

Mean Prediction : We took all 30/100 predictions and averaged them to get a center output for each input. The mean achieved here was used as the final prediction, being an expected value across these stochastic passes.

Min Prediction : We took all 30/100 and found the minimum of them to get the worst case output for each input. The value achieved here was used as the final prediction.

Standard Deviation of Predictions: We determined the uncertainty estimate represented by the standard deviation of these predictions. An output with a low standard deviation was interpreted as high confidence on behalf of the model while a bigger standard deviation indicated more uncertainty.

Lastly, we also used the minimum prediction between the 30 forward passes as an implement technique because we thought that its more conservative estimate of model performance(lower-bound) would help us in bettering our final output.

Results and Observations

The mean of the predictions from MC Dropout led to a best score of ~600 which is worse than our baseline GoogleNet model without dropout during inference. The minimum prediction metric did even worse, probably because dropout noise forced the minimum values away from a valid prediction.

To summarise, Although MC Dropout provided us a meaningful metric for uncertainty (measured with the declaration of standard deviation of predictions), it achieved no measurable gain in either predictive accuracy nor performance over our baseline GoogleNet model. This indicates that while MC Dropout is informative for confidence estimation (good to have), it does not necessarily provide a boost in predictive power.

Test Time Augmentation:

Test-time augmentation (TTA) is an uncertainty estimation technique that leverages multiple augmented versions of the same input during inference to assess model confidence.

Here's how it works:

Generating Augmented Samples: At inference (test) time, the original input (e.g., an image) is transformed multiple times using common augmentation techniques like rotation, flipping, scaling, or color adjustments. These transformations are similar to those often used in training for data augmentation but are applied only at inference here.

Making Predictions on Augmented Samples: The model makes a separate prediction for each augmented version of the input. This generates a set of predictions from slightly varied perspectives of the same data.

Estimating Uncertainty: By analyzing the variance or entropy among these predictions, we can estimate uncertainty. If all augmented versions yield similar

predictions, the model is considered confident (low uncertainty). However, if predictions vary widely across the augmented samples, it suggests that the model is uncertain about the input.

TTA is particularly useful in image classification and segmentation tasks, as it provides a way to test the model's robustness to small changes in the input, helping assess how confidently the model would perform on slightly altered or noisy data.

For the testing phase, we applied *test-time augmentation (TTA)* to improve our model's robustness and generalization. Our TTA pipeline included several transformations to simulate natural variations in the data, allowing the model to make predictions on multiple augmented versions of each test image. We applied ***random horizontal and vertical flips***, each with a 50% probability, to create *mirroring effects*. Additionally, we introduced ***random rotations*** of up to 15 degrees, along with ***random resized cropping*** to a 32x32 pixel size, with ***scaling*** between 0.8 and 1.0, to vary the positioning and aspect ratio of the images. Finally, we normalized the images using specific mean and standard deviation values, consistent with the model's training data distribution. By using these augmentations, we generated multiple slightly different views of each test image, enabling the model to make averaged or ensemble predictions.

Deep Ensembling Methods

In our experimentation, we applied bagging techniques with models like ResNet18 and ResNet50. However, these models resulted in poor accuracies, which limited their effectiveness for our task. We also tested GoogLeNet, but found that it required significantly more computation time, making it impractical for our current setup.

References -

- [1]<https://www.sciencedirect.com/science/article/pii/S2667096821000136/pdf?md5=158ada0135991ef5bb0b940fd7083a57&pid=1-s2.0-S2667096821000136-main.pdf>
- [2]<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9060995/>

- [3]https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf
- [4]<https://arxiv.org/pdf/2201.03545>
- [5]<https://arxiv.org/pdf/1608.06993>
- [6]<https://github.com/liuzhuang13/DenseNet>
- [7]<https://arxiv.org/pdf/1409.4842>
- [8]<https://github.com/weiaicunzai/pytorch-cifar100/blob/master/README.md>
- [9][Uncertainty-informed deep learning models enable high-confidence predictions for digital histopathology | Nature Communications](#)